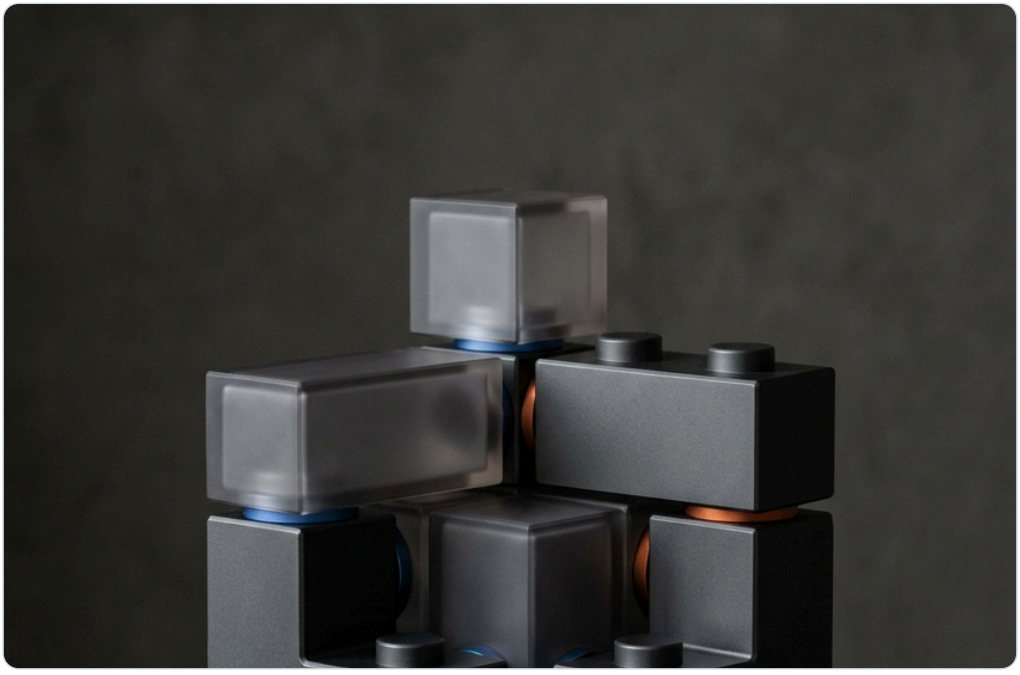
ARQUITETURA DE SISTEMAS AGÊNTICOS

LegoLoop

O Mega Pipeline da IA Modular

A Engenharia Determinística dos 5 Sub-Loops, Contexto Estéril e Governança de Processos com IA



Heverton & Ecossistema LegoLoop

06/11/2026 · LegoLoop Foundation

SOBRE ESTA OBRA

Este livro documenta a arquitetura, ontologia e implementação do Mega Pipeline do LegoLoop — o orquestrador mestre determinístico para desenvolvimento de software auxiliado por inteligência artificial. A obra formaliza o conceito de processo na engenharia agêntica como uma máquina de estados finita, governada e orientada a contratos, eliminando o caos probabilístico de execuções soltas.

Percorremos a esteira completa dividida em três níveis canônicos: o nível macro (a ontologia de processos e o paradigma fractal), o nível meso (o fluxo dos cinco sub-loops com isolamento hermético via Orchestrix) e o nível micro (o detalhamento das etapas internas, seus contratos de entrada, processamento, peças acopladas, gates mecânicos e handoffs imutáveis).

A obra aborda também a sextilha canônica de peças e a cascata de governança em quatro estágios, oferecendo aos arquitetos, engenheiros e desenvolvedores o guia definitivo para construir esteiras industriais previsíveis, de altíssimo rigor e livre de alucinações.

Documento gerado deterministicamente a partir das fontes citadas no próprio texto, via pandoc + typst. Toda afirmação técnica desta obra é rastreável a um arquivo nomeado no capítulo correspondente.

Data da compilação: 06/11/2026 · LegoLoop Foundation

Sumário

Como ler este livro **2**
A quem este livro se destina 2
A estrutura da obra 2
As convenções visuais 3
PARTE I — NÍVEL MACRO: ONTOLOGIA E PROCESSO **4**
1. O Conceito Formal de Processo na Engenharia de IA 4
2. A Arquitetura Fractal do LegoLoop 6
3. A Sextilha Canônica de Peças 7
Rastreabilidade 8
PARTE II — NÍVEL MESO: O MEGA PIPELINE E A ORCHESTRIX **9**
1. Visão Panorâmica dos Cinco Sub-Loops 9
2. A Orchestrix Cirúrgica e o Handoff File-Based 10
3. Roteamento Cognitivo e Effort Clamping 11
4. As Sete Leis Invioláveis da Esteira 12
Rastreabilidade 13
PARTE III — NÍVEL MICRO: ANATOMIA DOS CINCO SUB-LOOPS **14**
1. Sub-Loop 1: Concepção & Alinhamento de Intenção 14
2. Sub-Loop 2: Engenharia Reversa & Design Pass *(Condicional)* 15
3. Sub-Loop 3: Engenharia & Tríade de Planejamento 16
4. Sub-Loop 4: Construção TDD por Épico *(Orchestrix)* 18
5. Sub-Loop 5: Homologação E2E, Recompilação & Fechamento 20
Rastreabilidade 21
PARTE IV — TRANSVERSAL: TEMPLATES E COMPOSIÇÃO FRACTAL **22**
1. A Cascata Canônica em Quatro Estágios 22
2. A Composição Fractal via CLI: O Comando `compor` 23
3. O Ciclo de Vida dos Cinco Verbos de Processo 24
Rastreabilidade 25
APÊNDICES: RASTREABILIDADE, GLOSSÁRIO E ESTADO HONESTO **26**
Apêndice A — Matriz de Rastreabilidade Factual 26
Apêndice B — Glossário Terminológico 27
Apêndice C — Estado Honesto do Ecossistema 28

Como ler este livro

A quem este livro se destina

Esta obra foi projetada para três perfis centrais da engenharia contemporânea:

1. **Arquitetos de Sistemas e Agentes:** que necessitam desenhar fluxos de trabalho autônomos com garantias matemáticas de não colisão, orçamentação estrita de tokens e eliminação de vazamento de contexto (*Zero Context Bloat*). Comece pelo Capítulo 1 e siga para o Capítulo 2.
2. **Engenheiros de Plataforma e DevOps:** responsáveis por integrar ferramentas, MCPs, scripts determinísticos e esteiras de CI/CD à governança agêntica. Dê foco especial ao Capítulo 3 e às convenções de contratos no Capítulo 4.
3. **Desenvolvedores Full-Stack e Especialistas de TDD:** que operam o LegoLoop Studio e a interface de linha de comando no dia a dia. Explore detalhadamente as fichas técnicas do Capítulo 3 e o caderno prático no Capítulo 4.

A estrutura da obra

O livro organiza o conhecimento técnico em três camadas de profundidade analítica:



Cada nível responde de forma consistente a três perguntas fundamentais: * **Qual é o contrato?** Quais arquivos entram, quais saem e qual regra aprova a transição. * **Qual é o motor?** Quais cérebros cognitivos (LLMs) ou scripts locais realizam o trabalho. * **Qual é a segurança?** Como o isolamento estéril e os Stop-Gates humanos protegem a integridade do código.

As convenções visuais

O livro adota elementos gráficos padronizados para acelerar a leitura e evidenciar decisões:

> **PAINEL DE GOVERNANÇA & REGRAS CRÍTICAS**
>
> Caixas com esta barra lateral azul representam regras inquebráveis, invariantes de arquitetura ou alertas de segurança. Quando um ponto do sistema exigir parada para inspeção humana, ele será destacado dessa forma.

- **Caixas Escuras (`#no`):** Representam etapas puramente mecânicas e determinísticas (scripts locais, verificações de arquivos, linters e compilação de binários) que rodam sem qualquer uso de inteligência artificial.
- **Caixas Claras (`#no-claro`):** Representam etapas cognitivas onde modelos de linguagem são acionados sob limites estritos de raciocínio (*effort clamping*).
- **Fichas Técnicas (`#ficha`):** Tabelas sintéticas que abrem cada seção apresentando a carteira de identidade daquele componente.

PARTE I — NÍVEL MACRO: ONTOLOGIA E PROCESSO

1. O Conceito Formal de Processo na Engenharia de IA

Na ciência da computação clássica e na engenharia de sistemas de missão crítica, a definição de um **Processo** é rigorosa:

> **DEFINIÇÃO AXIOMÁTICA DE PROCESSO**
>
> Um processo é uma sequência finita, ordenada e governada de transições de estado, que transforma entradas definidas em saídas verificáveis, operando sob regras explícitas, com recursos previamente alocados e sob critérios determinísticos de validação e parada.

Quando a inteligência artificial generativa foi introduzida no desenvolvimento de software, a indústria cometeu o erro de tratá-la como um chat interativo contínuo. Em conversas longas e desestruturadas, os modelos sofrem de alucinação cumulativa, esquecimento de diretrizes anteriores e poluição descontrolada de contexto (*context bloat*).

O LegoLoop nasce para refundar a engenharia de software com IA sobre bases fabris determinísticas. Em vez de prompts soltos, todo trabalho é encapsulado em uma **máquina de estados finita** representada por um Grafo Acíclico Dirigido (DAG).

> **Um processo é uma máquina de estados finita: cada nó transforma entradas declaradas em saídas verificáveis, sem depender de memória mágica.**



O Monólito Tradicional versus a Esteira Modular LegoLoop

Natureza	**Máquina de Estados Finita e Auditável**
Grafo Base	Directed Acyclic Graph (DAG v2) em JSON Schema
Garantia de Parada	Proibição estrita de ciclos infinitos (ErrCiclo)
Handoff	Exclusivamente mediado por sistema de arquivos (SSOT)
Ambiente	Workspaces temporários herméticos criados pelo Orchestrix

Os Cinco Pilares de Governança do Processo

1. **Sequência Finita e Ordenada:** Cada processo possui um início formal demarcado por um gatilho (`trigger`) e um término inequívoco. A ordem de execução não é estocástica: o resolvedor topológico do motor garante que uma etapa só execute após todos os seus nós predecessores declarados em `depends_on` terem sido concluídos com sucesso.
2. **Transformação de Estados Auditável:** Um nó nunca "acha" o que deve fazer. Ele recebe uma lista fechada de arquivos de entrada (`inputs`), aplica uma transformação controlada e materializa um artefato no disco (`output.artifact`).
3. **Regras Explícitas e Ausência de Premissas Ocultas:** A IA é contratualmente proibida de inferir comportamento arquitetural em silêncio. Toda decisão deve estar ancorada nos documentos da tríade de planejamento.

4. **Recursos Rigidamente Alocados:** Cada bloco do processo declara com antecedência sua cota de recursos: qual modelo cognitivo será acionado, qual o teto de tokens de saída (`max_output_tokens`), qual o limite de tempo (`timeout_sec`) e quais ferramentas locais estão liberadas.
5. **Critérios Determinísticos de Validação:** A aprovação de uma etapa não depende da avaliação subjetiva do próprio modelo. Ela é julgada por expressões booleanas matemáticas puras compiladas em DSL (Domain Specific Language), tais como conferência de código de retorno (`exit_code == 0`) e existência física do arquivo (`file_exists`).

---

2. A Arquitetura Fractal do LegoLoop

Uma das propriedades mais poderosas da engenharia do LegoLoop é a sua **natureza fractal e homotética**. A mesma estrutura conceitual e contratual que rege a esteira inteira se repete em cada uma de suas subdivisões:



- **Mega Pipeline (A Fábrica Completa):** Encadeia a visão global de ponta a ponta. Ele transforma uma ideia bruta ou solicitação humana em um produto de software compilado, testado em múltiplos dispositivos e documentado com rastreabilidade de sessão.
- **Sub-Loops (As Linhas de Montagem):** São especializações funcionais do processo (ex.: Concepção, Engenharia Reversa, Planejamento BDD, Construção TDD e Homologação). Cada sub-loop tem vida própria: pode ser invocado de forma autônoma ou encadeado como uma macro-peça.
- **Loops e DAGs (O Contrato Isomórfico):** No nível de dados e código, **não há diferença estrutural entre um loop simples e um mega pipeline**. Ambos são manifestos JSON aderentes ao mesmo esquema formal (`https://legoloop.dev/schemas/dag.v1.json`). O Mega Pipeline é apenas um loop cujos nós referenciam outros loops.

- **Nós e Etapas (`dag.Node`):** É a menor partícula operacional. Um nó nasce em um diretório temporário, recebe os artefatos de entrada, executa as ferramentas atribuídas, submete seu resultado ao crivo mecânico de um gate e encerra sua existência sem deixar resíduos de memória.

3. A Sextilha Canônica de Peças

Para que uma esteira seja flexível e componível, as etapas do processo utilizam um catálogo ontológico padronizado em seis naturezas de peças:

Natureza da Peça	Formato Nativo	Função Primária no Fluxo	Motor de Execução
1. Skill	Markdown (`SKILL.md`)	Prompt especialista com heurísticas e raciocínio de IA	`Orchestrix`
2. Script	PowerShell / Shell / Go	Transformação determinística de dados e arquivos locais	`Runtime Local`
3. Hook	Configuração JSON	Interceptação de ciclo de vida pré e pós-execução	`Harness Hooks`
4. MCP	Configuração JSON (`.mcp.json`)	Ferramental vivo externo (navegador, banco de dados)	`Protocolo MCP`
5. Gate	Expressão booleana em DSL	Validação mecânica binária e aprovação de transição	`internal/gate`
6. Loop	Manifesto JSON (`.loop.json`)	Encadeamento de nós em DAG para reúso modular	`dag.Parse`

> **EQUILÍBRIO COGNITIVO VS. DETERMINÍSTICO**
>
> O segredo de uma esteira robusta está no casamento entre peças cognitivas (Skills) e peças determinísticas (Scripts e Gates). A inteligência artificial só deve ser chamada para raciocínio, escrita e síntese; tudo o que puder ser medido por um compilador, formatador ou script local deve ser executado fora da LLM para economizar tokens e garantir tolerância zero a falhas.

Rastreabilidade

- Código do grafo e contrato de nós: `internal/dag/types.go` e `internal/dag/parse.go`.
- Validação de ciclos infinitos: `internal/dag/cycle.go`.
- Definições de templates canônicos: `docs/templates/README.md` e `docs/templates/TEMPLATE_PIECE_LOOP.json`.

PARTE II — NÍVEL MESO: O MEGA PIPELINE E A ORCHESTRIX

1. Visão Panorâmica dos Cinco Sub-Loops

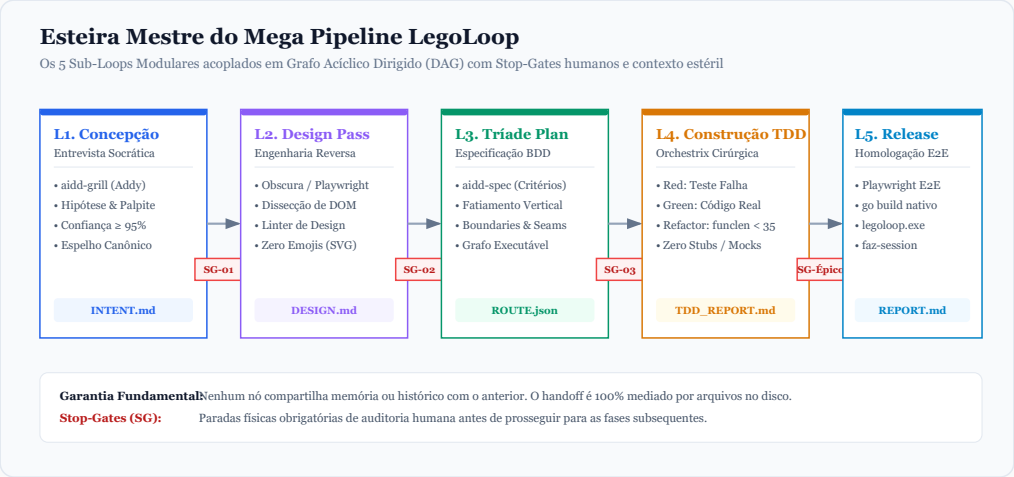
O **Mega Pipeline** organiza o ciclo de vida completo de engenharia de software em cinco sub-loops lineares e complementares. Nenhum sub-loop avança para o próximo sem que seu artefato terminal tenha sido aprovado por um portão de qualidade:



1. **Loop 1 — Concepção & Alinhamento:** Transforma o pedido do usuário em uma declaração clara de intenção (`INTENT.md`) através de uma entrevista socrática.
2. **Loop 2 — Engenharia Reversa & Design Pass:** Inspeciona referências visuais ou aplicações legadas, extraindo tokens limpos e wireframes técnicos (`DESIGN.md`).
3. **Loop 3 — Especificação & Tríade de Planejamento:** Formaliza a engenharia em critérios BDD executáveis (`SPEC.md`), quebra o trabalho em tickets (`PLAN.md`) e compila a rota acíclica (`ROUTE.json`).
4. **Loop 4 — Construção TDD por Épico:** Implementa o código em ciclo estrito de testes unitários (Red ➔ Green ➔ Refactor), módulo a módulo, sem tolerância a stubs.

5. **Loop 5 — Homologação E2E & Fechamento:** Audita o sistema com navegadores reais headless, recompila os binários de produção, consolida métricas de tokens e gera o relatório final.

> **Nenhum bloco herda o contexto do anterior. A Orchestrix cria e destrói diretórios estéreis em milissegundos, garantindo Zero Context Bloat.**



A Esteira Mestre dos Cinco Sub-Loops do Mega Pipeline

2. A Orchestrix Cirúrgica e o Handoff File-Based

O maior ofensor de custos e alucinações em sistemas agênticos convencionais é o compartilhamento indiscriminado de histórico de conversação entre tarefas. Quando um agente recebe centenas de mensagens acumuladas de etapas passadas, o ruído semântico destrói a atenção do modelo.

O motor **Orchestrix** do LegoLoop resolve esse problema impondo o isolamento cirúrgico por bloco:

Mecanismo	Diretório temporário e isolamento hermético

Contrato de Entrada	inputs: declaração estrita de arquivos consumidos
Função de Transferência	transferInputs(): copia apenas o necessário
Contrato de Saída	output.artifact: gravação no diretório pai
Limpeza de Memória	defer os.RemoveAll(): zero lixo residual no SO

O Ciclo de Execução Estéril de um Sub-Loop

Quando a Orchestrix é instruída a executar um nó do tipo `"type": "subloop"`, o código em `internal/orchestrix/subloop.go` executa o seguinte protocolo de seis passos:

1. **Geração do Workspace Estéril:** O runtime chama `os.MkdirTemp("", "legoloop-subloop-*")`, instanciando uma pasta limpa no disco.
2. **Transferência Cirúrgica de Entradas:** A função `transferInputs` inspeciona a lista de arquivos declarados em `inputs` e copia apenas esses arquivos para dentro da pasta temporária. Nenhum arquivo estranho é incluído.
3. **Execução Isolada do Sub-Grafo:** O sub-grafo interno é parsed e executado pelo `runner.Run`. As etapas internas consomem e produzem dados exclusivamente dentro daquela bolha.
4. **Avaliação dos Gates Mecânicos:** O gate de saída de cada etapa é testado localmente. Se houver falha, o processo é imediatamente interrompido.
5. **Extração do Artefato Final:** A função `finalizeSubloop` copia o arquivo terminal contratado (`sub_artifact`) de volta para o diretório do projeto principal, preservando seu hash e integridade.
6. **Destruição do Ambiente:** O diretório temporário é totalmente apagado via `os.RemoveAll()`, garantindo que resíduos de compilação ou arquivos intermediários não contaminem as etapas futuras.

---

3. Roteamento Cognitivo e Effort Clamping

Nem toda tarefa exige o mesmo poder de fogo computacional. Enviar a correção de um erro de lint para um modelo de raciocínio profundo de última geração é um desperdício financeiro inaceitável; da mesma forma, confiar a concepção da arquitetura de um sistema complexo a um modelo leve leva a colapsos de lógica.

O Mega Pipeline introduz a matriz de **Roteamento Cognitivo com Clamping de Esforço**:

Nível de Tarefa	Escopo e Complexidade	Modelo Cognitivo Sugerido	Flag de Esforço
Épico (Macro)	Arquitetura, contratos BDD, DAG e modelagem	`Claude 3.7 Sonnet`, `Gemini 2.0 Pro`	`--effort high`
Micro-ticket	Módulo funcional, suíte de testes unitários	`Claude 3.5 Sonnet`, `Gemini 2.0 Flash`	`--effort med`
Nano-ticket	Correção de lint, tipo TypeScript, assert	`Gemini Flash-Lite`, `Ollama Local 8B`	`--effort low`

> **REGRA DE CLAMPING DE TOKENS**
>
> A flag `--effort` define o teto de tokens de raciocínio interno (thinking budget) alocados pela API. Em nano-tickets, o modelo é instruído a responder instantaneamente sem divagações, preservando a cota semanal para o que realmente importa: o design arquitetural e os contratos de teste.

4. As Sete Leis Invioláveis da Esteira

O Mega Pipeline opera debaixo de sete mandamentos mandatórios que nenhum agente ou executor tem autorização para violar:

1. **Zero Context Bloat:** Nenhum bloco herda o histórico conversacional de blocos anteriores.
2. **Stop-Gates de Auditoria Humana:** Paradas obrigatórias entre concepção, planejamento e ao término de cada Épico de código.
3. **A Tríade Inegociável:** O código só pode começar se existirem no disco a `SPEC.md`, o `PLAN.md` e a rota executável `ROUTE.json`.
4. **Provider-Awareness:** Roteamento de modelos adaptado ao ambiente local do desenvolvedor (`9Router`, `FreeLLMAPI` ou chave direta).

5. **TDD Estrito & Zero Stubs:** O teste unitário deve falhar comprovadamente antes do código de produção. Proibição absoluta de funções vazias com `TODO` ou `pass`.
6. **Padronização ISO de Nomenclatura:** Todos os artefatos de documentação seguem o prefixo `YYYY-MM-DD_<NOME>.<ext>`.
7. **Executor Silencioso:** Respostas concisas no chat e pensamento interno telegráfico no padrão Caveman.

Rastreabilidade

- Implementação do isolamento de sub-loops: `internal/orchestrix/subloop.go`.
- Runner determinístico do DAG: `internal/runner/runner.go`.
- Regras de conduta de agentes: `AGENTS.md` e `docs/manuals/2026-11-01_MANUAL_MEGA_PIPELINE_LEGOLOOP.md`.

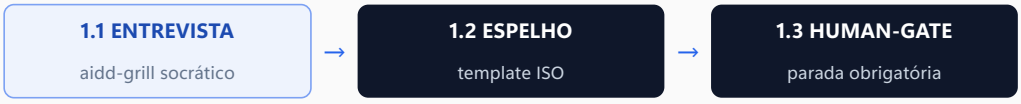
PARTE III — NÍVEL MICRO: ANATOMIA DOS CINCO SUB-LOOPS

Esta seção detalha o nível atômico do Mega Pipeline. Cada um dos cinco sub-loops é dissecado em suas etapas operacionais, apresentando os contratos rigorosos de entrada, processamento, ferramental acoplado, validação mecânica e handoff imutável.

1. Sub-Loop 1: Concepção & Alinhamento de Intenção

O primeiro sub-loop tem por objetivo extrair a verdadeira necessidade do usuário, eliminando a ambiguidade natural da linguagem humana antes de qualquer especificação técnica.

Papel	**Alinhamento socrático de escopo e premissas**
Disparo	Linguagem natural do usuário via chat ou trigger manual
Entrada	Briefing inicial ou descrição da demanda
Saída Principal	docs/intents/YYYY-MM-DD_INTENT_<TEMA>.md
Portão de Saída	STOP-GATE HUMANO 01 (Aprovação da Intenção)



Etapa 1.1 — Entrevista Socrática (Briefing Ativo)

- **Consome:** Linguagem natural do usuário via chat ou comando CLI.
- **Peças:** Skill `aidd-grill` + Hook de telemetria de tokens.
- **Processamento:** O agente assume uma postura socrática empática e incisiva, formulando uma pergunta por vez. Cada intervenção traz quatro marcadores formais: `HIPÓTESE`, `CONFIANÇA [X%]`, `PERGUNTA` afuniladora e `PALPITE` sugerido. O interrogatório se encerra quando a confiança atinge $\geq 95\%$ ou o modelo prevê com certeza as próximas respostas.
- **Gate:** Confiança calculada $\geq 95\%$ ou encerramento socrático acordado.

- **Handoff:** Transfere a estrutura de hipóteses validadas em memória para a Etapa 1.2.

Etapa 1.2 — Geração do Espelho Canônico e Persistência

- **Consome:** Dados consolidados da Etapa 1.1.
- **Peças:** Script gerador de template ISO + Gate mecânico.
- **Processamento:** Materializa o documento formal estruturado em seis seções fixas: Resultado Esperado, Quem se Beneficia, Por que Agora, Critérios de Sucesso, Restrições Inegociáveis e Fora de Escopo.
- **Gate:** Expressão DSL: `file_exists('docs/intents/YYYY-MM-DD_INTENT_<TEMA>.md')`.
- **Handoff:** Grava o arquivo físico persistido no disco.

Etapa 1.3 — Human-Gate 01 (Validação da Intenção)

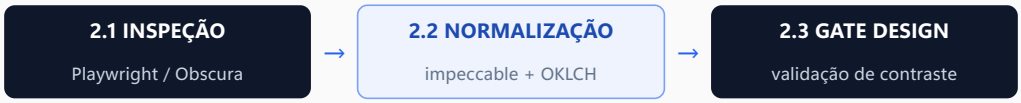
- **Consome:** Documento de intenção gerado na Etapa 1.2.
- **Peças:** Controle interativo `human-gate`.
- **Processamento:** O runtime entra em pausa física obrigatória. Exibe o espelho canônico e aguarda o comando humano (`[1] seguir`, `[2] personalizar`, `[0] cancelar`).
- **Gate:** Confirmação explícita do usuário.
- **Handoff:** Destrava a execução para o Sub-Loop 2 ou Sub-Loop 3.

---

2. Sub-Loop 2: Engenharia Reversa & Design Pass *(Condicional)*

Acionado quando a demanda envolve recriar telas existentes, reconstruir interfaces legadas ou extrair design systems de produtos de terceiros.

Papel	**Captura de runtime, dissecção DOM e tokens de design**
Disparo	Condicional: presença de URL, mock, binário ou tela legada
Entrada	URL ou arquivos de interface existentes
Saída Principal	DESIGN.md e TOKENS.css
Portão de Saída	STOP-GATE HUMANO 02 (Validação Visual)



Etapa 2.1 — Inspeção de Runtime & Captura DOM

- **Consome:** URL de referência externa, binário compilado ou mock de tela.
- **Peças:** MCP `playwright`, MCP `chrome-devtools` ou motor `obscura` em Rust.
- **Processamento:** Inicia o navegador em ambiente isolado, captura a árvore DOM semântica, calcula os estilos reais de CSS e realiza capturas de tela multi-viewport.
- **Gate:** `exit_code == 0 && file_exists('scratch/raw_dom.json')`.
- **Handoff:** Grava o payload JSON da estrutura da tela em `scratch/raw_dom.json`.

Etapa 2.2 — Normalização e Extração de Tokens

- **Consome:** `scratch/raw_dom.json` da Etapa 2.1.
- **Peças:** Skill `impeccable` + Script de paleta OKLCH.
- **Processamento:** Erradica 100% de emojis amadores, extrai tokens semânticos de espaçamento, tipografia e cores, e valida contrastes em conformidade com as normas WCAG.
- **Gate:** Zero emojis detectados e contraste de cores aprovado pelo script de validação.
- **Handoff:** Grava os artefatos `DESIGN.md` e `TOKENS.css`.

Etapa 2.3 — Gate de Design & Human-Gate 02

- **Consome:** `DESIGN.md` e `TOKENS.css`.
- **Peças:** Controle `human-gate` + Script linter de design.
- **Processamento:** Apresentação visual da paleta e layout propostos para aprovação do desenvolvedor.
- **Gate:** Aval humano formal registrado no Studio ou terminal.
- **Handoff:** Libera os tokens aprovados para a especificação do Sub-Loop 3.

---

3. Sub-Loop 3: Engenharia & Tríade de Planejamento

É o coração arquitetural do LegoLoop. Ele converte a intenção humana e o design visual em um contrato de engenharia matematicamente executável.

Papel	**Especificação BDD, decomposição hierárquica e rota DAG**
Disparo	Conclusão e aprovação dos Sub-Loops 1 e 2
Entrada	INTENT.md e opcionalmente DESIGN.md
Saída Principal	A Tríade: SPEC.md, PLAN.md e ROUTE.json
Portão de Saída	STOP-GATE HUMANO 03 (Auditoria da Tríade)



Etapa 3.1 — Especificação BDD (SPEC)

- **Consome:** `docs/intents/INTENT.md` (+ `DESIGN.md` se presente).
- **Peças:** Skill `aidd-spec`.
- **Processamento:** Redige as premissas técnicas explícitas, define as fronteiras em 3 níveis (*Sempre Fazer*, *Perguntar Antes*, *Nunca Fazer*), demarca as costuras de teste (*Test Seams*) e formaliza os critérios de aceite binários `AC-E{N}.{M}` no formato rigoroso Dado/Quando/Então.
- **Gate:** `file_exists('docs/specs/YYYY-MM-DD_SPEC_<TEMA>.md') && has_binary_criteria(...)`.
- **Handoff:** Grava o documento de especificação técnica no disco.

Etapa 3.2 — Decomposição Hierárquica em Tickets (PLAN)

- **Consome:** `docs/specs/SPEC.md` da Etapa 3.1 *(Dependência estrita)*.
- **Peças:** Skill `aidd-tickets`.
- **Processamento:** Fatiamento vertical da especificação em uma hierarquia de tickets: Épicos (>20k tokens de contexto), Micro-tickets (4k a 20k tokens para TDD pontual) e Nano-tickets (<4k tokens para correções de lint).
- **Gate:** `file_exists('docs/plans/YYYY-MM-DD_PLAN_<TEMA>.md')`.
- **Handoff:** Grava o plano unificado com tabela estruturada.

Etapa 3.3 — Compilação da Rota Executável (ROUTE)

- **Consome:** `docs/plans/PLAN.md` e `docs/specs/SPEC.md`.
- **Peças:** Skill `aidd-orchestrate-pipeline-creator` + Script validador JSON Schema.

- **Processamento:** Converte a tabela legível do plano em um mapa formal JSON com declaração de IDs únicos, dependências, modelo roteado (`code-pro`, `code-fast`, `code-nano`), esforço (`--effort`) e comandos de verificação mecânica.
- **Gate:** Validação contra o schema oficial (`exit_code == 0`).
- **Handoff:** Grava `docs/routes/YYYY-MM-DD_ROUTE_<TEMA>.json`.

Etapa 3.4 — Human-Gate 03 (Auditoria da Tríade)

- **Consome:** Tríade completa (`SPEC.md` + `PLAN.md` + `ROUTE.json`).
- **Peças:** Controle `human-gate`.
- **Processamento:** Parada obrigatória antes de tocar em qualquer arquivo da aplicação. O desenvolvedor valida se o plano reflete as decisões arquiteturais corretas.
- **Gate:** Aprovação explícita do usuário.
- **Handoff:** Destrava a inicialização da construção no Sub-Loop 4.

---

4. Sub-Loop 4: Construção TDD por Épico *(Orchestrix)*

É o sub-loop onde o código real de software é escrito. Ele executa cada micro-ticket isoladamente dentro de um ciclo rigoroso de Test-Driven Development.

Papel	**Construção de código de produção orientado a testes unitários**
Disparo	Aprovação formal da Tríade de Planejamento
Entrada	ROUTE.json e âncoras da SPEC.md
Saída Principal	Código de produção testado e TDD_REPORT.md
Portão de Saída	STOP-GATE POR ÉPICO (Revisão Humana de Código)



Etapa 4.1 — Ciclo RED (Teste Falhando Primeiro)

- **Consome:** Seção âncora do critério de aceite na SPEC (`spec_anchor`) e arquivo de teste alvo.
- **Peças:** Skill `aidd-tdd` + Script executor de testes locais (`go test`).
- **Processamento:** O agente implementa o teste unitário estrito reproduzindo fielmente o cenário do critério BDD.

- **Gate:** O teste **deve falhar comprovadamente** com erro de asserção funcional (`exit_code != 0`). Uma falha de compilação invalida o teste.
- **Handoff:** Grava o código do teste e o log de falha em `scratch/test_red.log`.

Etapa 4.2 — Ciclo GREEN (Implementação Mínima de Produção)

- **Consome:** Teste falhando gerado na Etapa 4.1 *(Dependência estrita)*.
- **Peças:** Skill `implementacao` + Script de compilação.
- **Processamento:** Escreve o código de produção mínimo e limpo para fazer o teste unitário passar. Proibição absoluta de stubs, mocks falsos, `TODO` ou `pass`.
- **Gate:** A suíte de teste daquele micro-ticket deve retornar código de sucesso (`exit_code == 0`).
- **Handoff:** Salva o código de produção funcional no workspace do ticket.

Etapa 4.3 — Ciclo REFACTOR & Quality Gates

- **Consome:** Código funcional aprovado na Etapa 4.2.
- **Peças:** Scripts locais de qualidade: `go vet ./...`, `funclen` (nenhuma função > 35 linhas), `gofmt -l .`.
- **Processamento:** Limpeza de redundâncias, padronização idiomática de código e redução da complexidade ciclomática.
- **Gate:** Zero violações de linter e saída vazia no verificador de formatação.
- **Handoff:** Consolida os commits locais do micro-ticket no branch do Épico.

Etapa 4.4 — Human-Gate por Épico

- **Consome:** Conjunto de micro-tickets finalizados do Épico ativo.
- **Peças:** Controle `human-gate`.
- **Processamento:** Pausa obrigatória ao fechar um Épico. Apresenta o sumário de testes verdes e commits para auditoria humana antes de avançar para o próximo Épico.
- **Gate:** Aprovação explícita do desenvolvedor.
- **Handoff:** Encaminha para o próximo Épico ou, ao término de todos, para o Sub-Loop 5.

5. Sub-Loop 5: Homologação E2E, Recompilação & Fechamento

O último sub-loop garante que o sistema construído opera em perfeita harmonia no mundo real, atualiza os binários executáveis e encerra a rastreabilidade da sessão.

Papel	**Homologação ponta a ponta, compilação de binários e fechamento**
Disparo	Conclusão e aprovação de 100% dos Épicos de construção
Entrada	Código de produção completo no repositório
Saída Principal	Binário executável no PATH e REPORT.md
Portão de Saída	STOP-GATE FINAL (Homologação e Entrega)



Etapa 5.1 — Homologação E2E Multi-Viewport & Smoke Test

- **Consome:** Aplicação viva rodando na porta local do ambiente de desenvolvimento.
- **Peças:** Skill `aidd-homologue` + MCP `playwright`.
- **Processamento:** Navega autonomamente por todos os fluxos críticos de usuário em resoluções mobile, tablet e desktop, auditando erros de console HTTP e regressões visuais.
- **Gate:** Zero erros de console HTTP e suíte E2E 100% verde.
- **Handoff:** Grava o relatório de execução e telas de evidência em `scratch/e2e/`.

Etapa 5.2 — Recompilação Obrigatória de Binários

- **Consome:** Código-fonte final aprovado no repositório.
- **Peças:** Script local de compilação: `go build -o bin/legoloop.exe ./cmd/legoloop`.
- **Processamento:** Compilação nativa de release do executável para garantir que as alterações no motor Go estejam ativas no PATH do sistema operacional.
- **Gate:** `exit_code == 0 && file_exists('bin/legoloop.exe')`.
- **Handoff:** Atualiza o binário oficial no diretório `bin/`.

Etapa 5.3 — Consolidação de Relatório & Sincronização de Sessão

- **Consome:** Logs de execução, telemetria de tokens de todos os blocos e status dos gates.

- **Peças:** Skill `aidd-sessao` + Script CLI `faz-session`.
- **Processamento:** Consolida métricas de tokens de entrada e saída, custo financeiro aproximado, modelos utilizados e balanço de conformidade contra a SPEC inicial.
- **Gate:** `file_exists('docs/reports/YYYY-MM-DD_REPORT_<TEMA>.md')`.
- **Handoff:** Grava o relatório final consolidado em `docs/reports/` e salva o histórico da sessão.

Etapa 5.4 — Human-Gate Final (Entrega do Produto)

- **Consome:** Relatório final consolidado e binário compilado.
- **Peças:** Controle `human-gate`.
- **Processamento:** Apresentação formal de encerramento da entrega para o cliente ou líder técnico.
- **Gate:** Aceite final assinado pelo usuário humano.
- **Handoff:** Conclusão bem-sucedida do Mega Pipeline.

Rastreabilidade

- Esquema canônico de nós: `internal/dag/types.go`.
- Implementação do runner e compilação de gates: `internal/gate/gate.go`.
- Manual mestre dos sub-loops: `docs/manuals/2026-11-01_MANUAL_MEGA_PIPELINE_LEGOLOOP.md`.

PARTE IV — TRANSVERSAL: TEMPLATES E COMPOSIÇÃO FRACTAL

Os templates do LegoLoop não são sugestões estéticas ou formulários vazios; eles são os **contratos fundacionais** que garantem que o sistema opere como uma máquina de estados determinística. Se uma ferramenta ou modelo não consegue interpretar o formato de um template, aquele arquivo deixa de ser engenharia e vira mera decoração.

1. A Cascata Canônica em Quatro Estágios

Toda demanda que entra no ecossistema passa compulsoriamente por quatro estágios de refinamento sucessivo antes que a primeira linha de código de produção seja autorizada:



Estágio de Governança	Arquivo de Template	Função Primária	Validação no Motor
1. INTENT	`TEMPLATE_INTENT.md`	Registra a dor do usuário, hipóteses e premissas	`Human-Gate 01`
2. SPEC	`TEMPLATE_SPEC.md`	Define premissas, boundaries e critérios BDD `AC-XX`	`has_binary_criteria`
3. PLAN	`TEMPLATE_PLAN.md`	Fatiamento vertical hierárquico (Épicos ➔ Nano)	`Human-Gate 02`
4. ROUTE	`TEMPLATE_ROUTE.json`	Grafo acíclico executável em formato machine-readable	`dag.Parse` e Schema

> **O PLAN para os olhos humanos é o espelho exato da ROUTE para os nós de máquina. Divergência entre os dois é defeito impeditivo.**



A Tríade Canônica da Fabricação Autônoma

A REGRA DE ESPELHO INEGOCIÁVEL

O PLAN (para humanos) e a ROUTE (para as máquinas) são duas faces da mesma moeda. Eles compartilham obrigatoriamente o mesmo conjunto de identificadores de nós, as mesmas dependências causais, os mesmos limites de timeout e o mesmo roteamento cognitivo. Qualquer divergência entre o PLAN e a ROUTE constitui defeito impeditivo de compilação.

2. A Composição Fractal via CLI: O Comando `compor`

A grande inovação arquitetural do LegoLoop é a capacidade de transformar qualquer processo isolado em uma etapa acoplável de um processo maior.

Imagine que você fabricou um processo chamado `subloop-tdd` contendo o ciclo completo de Red-Green-Refactor. Quando você estiver desenhando um novo sistema corporativo, você não precisa redesenhar essas etapas:

Comando CLI	**legoloop compor <alvo> <subloop> [--pos N]**
Ação no DAG	Insere nó do tipo 'subloop' com ref ao loop filho
Prevenção de Ciclos	cycle.DetectCycle(): recusa loops recursivos
Isolamento de I/O	Separação de caminhos via sub_artifact
Versionamento	Elevação semântica automática de versão (ex: 1.0.0 -> 1.1.0)

Como a Composição Funciona nos Bastidores

1. **Validação Estrutural:** O comando verifica se ambos os loops existem no cofre (`~/.aidd/loops/` ou repositório) e confere as assinaturas de entrada e saída.
2. **Prevenção Mecânica de Ciclos:** O resolvedor de grafos simula a injeção. Se o loop `A` compuser o loop `B`, e o loop `B` já contiver referências ao loop `A`, o sistema aborta com erro fatal `ErrCiclo`.
3. **Mapeamento de Artefatos com `sub_artifact`:** Para impedir que o artefato final do sub-loop colida com o artefato do pipeline pai (um bug clássico em geradores de código), o motor grava o campo `sub_artifact`. O sub-loop opera em sua pasta própria (`out/<subloop>/`) e a Orchestrix entrega apenas o resultado final no destino do pai.

---

3. O Ciclo de Vida dos Cinco Verbos de Processo

O LegoLoop padronizou a gestão de processos no ecossistema através de cinco verbos de comando universais, disponíveis tanto na CLI quanto na interface visual do Studio:

Verbo Operacional	Comando CLI	Papel no Ciclo de Vida
Fabricar	`legoloop fabricar "<nome>"`	Cria um processo novo do zero via cascata socrática

Verbo Operacional	Comando CLI	Papel no Ciclo de Vida
Compor	`legoloop compor "<alvo>" "<subloop>"`	Acopla um processo como sub-etapa de outro
Listar	`legoloop listar`	Audita o inventário de processos, versões e vínculos
Refinar	`legoloop refinar "<nome>"`	Substitui atomicamente um nó defasado por um melhor
Remover	`legoloop remover "<nome>"`	Desvincula um processo com garantia de rollback seguro

> **CANONIZAÇÃO E HASHING SHA-256**
>
> Toda peça ou template importado ou fabricado passa imediatamente pelo crivo da canonização (legoloop canonize). O motor calcula a impressão digital SHA-256 de cada ativo, desduplicando cópias idênticas e vinculando a reputação daquela peça no cofre central do desenvolvedor.

Rastreabilidade

- Implementação do comando compor: `internal/compor/compor.go` e testes em `internal/cli/compor_binary_test.go`.
- Validação de sub-artefatos sem colisão: `internal/dag/subartifact_test.go`.
- Contratos e templates SSOT: `docs/templates/templates.go`.

APÊNDICES: RASTREABILIDADE, GLOSSÁRIO E ESTADO HONESTO

Apêndice A — Matriz de Rastreabilidade Factual

Para garantir a auditabilidade total desta obra, a tabela abaixo mapeia cada conceito arquitetural apresentado diretamente ao código-fonte ou documento normativo correspondente no repositório:

Conceito ou Mecanismo	Arquivo de Implementação no LegoLoop	Responsabilidade Factual
Contrato do Nó DAG	`internal/dag/types.go`	Estrutura do nó, modos de roteamento e handoffs
Parser e Schema	`internal/dag/parse.go`	Validação sintática contra `schemas/dag.v1.json`
Prevenção de Ciclos	`internal/dag/cycle.go`	Algoritmo de DFS que impede grafos circulares
Isolamento de Sub-Loop	`internal/orchestrix/subloop.go`	Workspaces temporários herméticos e `transferInputs`
Compilação de Gates	`internal/gate/gate.go`	Compilador de expressões booleanas em DSL
Composição de Loops	`internal/compor/compor.go`	Injeção de nós sub-loop e elevação de versão
Sub-Artefatos sem Colisão	`internal/dag/subartifact_test.go`	Isolamento de saídas entre loops pai e filho
Templates Canônicos	`docs/templates/README.md`	Modelos oficiais da sextilha de peças
Mega Pipeline Master	`loops/mega-pipeline.loop.json`	Manifesto canônico da esteira dos 5 sub-loops
Manual dos Sub-Loops	`docs/manuals/2026-11-01_MANUAL_MEGA_PIPELINE_LEGOLOOP.md`	Especificação operacional e stop-gates

Apêndice B — Glossário Terminológico

- **Cascata Canônica em 4 Estágios:** Sequência mandatória de planejamento (`INTENT` ➔ `SPEC` ➔ `PLAN` ➔ `ROUTE`) que precede a escrita de qualquer código de produção.
- **Context Bloat:** Degradação severa da qualidade de raciocínio de um modelo provocada pelo acúmulo desnecessário de históricos de mensagens e arquivos não relacionados no prompt.
- **Directed Acyclic Graph (DAG):** Estrutura matemática de dados unidirecional e sem laços recursivos, utilizada para orquestrar etapas de processos com dependências lineares ou paralelas.
- **Effort Clamping:** Imposição de um teto rígido de tokens de raciocínio interno (`--effort low|med|high`) a uma chamada de IA, prevenindo gasto excessivo de cota em tarefas simples.
- **Gate Mecânico:** Expressão booleana avaliada deterministicamente pelo motor local sem intervenção de IA (ex.: `exit_code == 0 && file_exists(...)`).
- **Handoff File-Based:** Transferência de estado entre etapas realizada exclusivamente através da gravação e leitura de arquivos físicos no disco, garantindo reprodutibilidade e isolamento.
- **Human-Gate (Stop-Gate):** Ponto de controle onde o motor paralisa fisicamente a execução e exige uma decisão humana explícita antes de liberar o próximo estágio.
- **Mega Pipeline:** Macro-orquestrador do ecossistema que conecta múltiplos sub-loops para levar uma demanda do briefing à entrega final de produção.
- **Orchestrix:** O motor hermético de execução do LegoLoop, responsável por criar workspaces temporários, filtrar entradas e destruir ambientes após o término de cada nó.
- **Sextilha Canônica:** O conjunto das seis naturezas padronizadas de peças reutilizáveis no LegoLoop: Skills, Scripts, Hooks, MCPs, Gates e Loops.
- **Sub-Loop:** Sub-grafo acíclico autocontido que encapsula uma linha de montagem especializada (ex.: ciclo TDD ou engenharia reversa).

Apêndice C — Estado Honesto do Ecossistema

Em consonância com a Lei da Honestidade de Rotulagem, declaramos formalmente o estado real e as limitações conhecidas da implementação na data de publicação desta obra:

Recurso / Componente	Status Atual	Evidência de Funcionamento	Limitação Conhecida
Validação de DAG	Concluído	100% verde em `go test ./...`	Suporta apenas grafos estáticos em tempo de carga
Isolamento Orchestrix	Concluído	Provado em testes temporários	No Windows, remoção de pasta pode exigir retentativas se aberta
Composição de Loops	Concluído	Provado no ticket TICK-03.4	O sub-loop reescreve apenas o gate do artefato final
Templates Canônicos V2	Concluído	Testado em `templates_test.go`	Scorer de reputação ainda avalia estrutura por regex
Execução com LLM Real	Operacional	Testado com Claude e 9Router	Exige chave configurada e conexão ativa de rede